

Metric-Semantic 3D Reconstruction of a Desktop Scene using Multi-View Geometry

CP260-2026 Final Project Report
Robot Perception

V Surya Kiran, K Devi Prasad, M Charmistha

Task: Oriented Bounding Box Estimation for Semantic Entities

Course: CP260 — Robotic Perception, 2026

Instructor: Prof. Amrutur Bharadwaj

Dataset: 16-frame posed RGB desktop scene (2560×1440)

May 4, 2026

Abstract

We present a lightweight, geometry-first metric-semantic reconstruction pipeline for estimating 3D Oriented Bounding Boxes (OBBs) of connector ports located on the back panel of a desktop PC tower, using only 16 posed RGB images captured at 2560×1440 resolution. Rather than relying on learned open-vocabulary detectors — which consistently fail on small, visually subtle objects such as VGA, ethernet, and power sockets — our approach combines precise manual 2D bounding-box annotations on carefully selected frames with classical multi-view triangulation. A grid-correspondence Direct Linear Transform (DLT) scheme reconstructs dense 3D point clouds for each connector type, and Oriented Bounding Boxes are recovered via Principal Component Analysis (PCA) augmented with physical depth priors. Validation against the provided VGA socket ground truth demonstrates sub-centimetre centre accuracy. The pipeline is modular, dependency-light, and executes end-to-end on Google Colab in under

Contents

1 Introduction

2 Dataset and Camera Model

2.1 Image Sequence

2.2 Camera Intrinsics

2.3 Camera Poses

2.4 Ground Truth (Validation)

3 Pipeline Overview

4 Methodology

4.1 Semantic Annotation

4.2 Multi-View Triangulation

4.3 OBB Fitting with Depth Prior

4.4 Output Format

5 Implementation Details

5.1 Module Structure

5.2 Key Hyperparameters

5.3 Colab Compatibility

6 Results and Validation

7 Novel View Synthesis

8 Conclusion

References

1 Introduction

Metric-semantic reconstruction refers to the joint recovery of 3D geometric structure and semantic object labels from a set of images captured around a scene. This capability underpins a wide range of modern robotics and augmented reality applications, including robot manipulation, autonomous navigation, and scene understanding for assistive systems.

In this project, we are provided with a static desktop scene featuring a PC tower and are tasked with localising specific connector ports — power socket, ethernet socket, and VGA socket — as 3D Oriented Bounding Boxes (OBBs) in a shared world coordinate frame. The challenge is considerable: the target objects are physically tiny (the VGA socket measures approximately $35 \times 12 \times 6$ mm), yet must be localised with centimetre-level accuracy in 3D. Furthermore, no depth sensor is available — only posed RGB images and a calibrated camera intrinsic matrix.

An initial attempt using GroundingDINO — an open-vocabulary object detector driven by text prompts — was unsuccessful. The model consistently failed to produce reliable detections of the VGA port inside cropped tower regions, regardless of confidence threshold. This is a well-documented limitation of large open-vocabulary models when applied to small, visually homogeneous objects with limited pixel coverage in the image. A CLIP-based sliding-window fallback similarly produced undifferentiated scores (29–33) across all classes, confirming that zero-shot learned detection is unreliable in this setting.

Consequently, we adopt a geometry-first strategy with three stages: (1) manually annotating tight 2D bounding boxes for each port on the frames that best reveal the back panel; (2) triangulating a dense 3D point cloud for each port from those annotations using all available camera view pairs; and (3) fitting an OBB using PCA augmented with physical depth priors for each connector type. This approach eliminates all learned-detection failure modes, requires only standard libraries (OpenCV, Open3D, NumPy), and produces accurate, interpretable results.

2 Dataset and Camera Model

2.1 Image Sequence

The dataset comprises 16 RGB frames captured by a camera moving around a stationary desktop PC tower mounted on a white pedestal in an office environment. The frames are named frame_000319.png through frame_000531.png and span a roughly 180° arc around the back panel of the machine. All images are captured at full resolution: 2560×1440 pixels (16:9 aspect ratio). The scene contains diverse backgrounds including chairs, monitors, and whiteboards, which provide rich texture for pose recovery but also create clutter that complicates automatic detection of small connectors.

2.2 Camera Intrinsics

The imaging system follows a standard pinhole camera model with zero lens distortion (undistorted images are provided). The calibrated intrinsic matrix K is:

$$K = \begin{bmatrix} 1477.010 & 0 & 1298.250 \\ 0 & 1480.442 & 686.820 \\ 0 & 0 & 1 \end{bmatrix}$$

The horizontal and vertical focal lengths are $f_x = 1477.010$ px and $f_y = 1480.442$ px respectively, and the principal point $(c_x, c_y) = (1298.250, 686.820)$ px lies close to the image centre as expected for a well-calibrated camera. These values were provided in `intrinsic.json` and were used directly in all projection and triangulation computations — replacing any heuristic focal-length estimates is critical for accurate 3D localisation.

2.3 Camera Poses

The file `poses.json` supplies 704 camera-to-world transformation matrices T_{c2w} indexed by integer frame number. Only the 16 entries corresponding to the captured images are used. Each matrix encodes the rigid-body transformation:

$$T_{c2w} = [R \mid t; 0^T \mid 1] \text{ where } R \text{ in } SO(3), t \text{ in } R^3$$

The world-to-camera transform required for image projection is obtained by inversion: $T_{w2c} = T_{c2w}^{-1}$. The Z-translation component of the poses consistently places the camera at approximately 0.83 m above the floor plane, which serves as a sanity check on the height of triangulated points. Poses were validated by reprojecting known scene features across frame pairs before running the full pipeline.

2.4 Ground Truth (Validation)

A ground-truth OBB for the VGA socket is provided for pipeline validation. The reference values are:

Parameter	Ground Truth Value
Centre (x, y, z) [m]	(0.2705, 0.2261, 0.8349)
Extent (w, h, d) [m]	(0.0354, 0.0118, 0.0061)
Physical size	35.4 mm × 11.8 mm × 6.1 mm
Centre Z (height above floor)	≈ 0.835 m

The Z-coordinate of the VGA socket centre (0.835 m) is consistent with the connector being mounted at approximately desk-panel height, confirming the metric scale of the provided coordinate frame.

3 Pipeline Overview

The full reconstruction pipeline proceeds through six sequential stages as illustrated below. Each stage is independently testable and produces intermediate outputs that can be inspected for correctness before proceeding to the next step.

Stage	Module	Description
1	<code>data_loader.py</code>	Load images, poses, and intrinsics; validate frame alignment
2	<code>semantic.py</code>	Manual 2D bounding box annotation on selected frames
3	<code>pose_estimation.py</code>	Grid-correspondence DLT triangulation to 3D point clouds

4	pose_estimation.py	MAD outlier removal from triangulated point clouds
5	pose_estimation.py	PCA + depth-prior OBB fitting
6	utils.py	JSON export + projection validation visualisation

Figure 1: End-to-end pipeline — Manual Annotation → Multi-View Triangulation → Outlier Removal (MAD) → OBB Fitting (PCA + depth prior) → Projection Validation → answers.json Export.

4 Methodology

4.1 Semantic Annotation

Each target semantic entity — power socket, ethernet socket, and VGA socket — is annotated with a 2D axis-aligned bounding box $[x_1, y_1, x_2, y_2]$ in the original 2560×1440 image coordinate system. Two frames were selected for annotation based on three criteria: (i) the I/O panel is directly visible and unobstructed, (ii) the camera is at a near-frontal angle relative to the back panel, and (iii) the frames are sufficiently separated in camera position to provide a strong triangulation baseline. Frames 471 and 496 satisfied all three criteria and were used as the primary annotation frames.

Annotation was performed manually using pixel coordinates read from the matplotlib axis display. Each bounding box tightly wraps the connector housing including its surrounding metal shield. The annotation quality directly governs the accuracy of the downstream triangulation — a 10-pixel error in annotation translates to approximately 0.7 mm of 3D positional error at the 0.83 m working depth of the scene.

Entity	Frame	x1	y1	x2	y2	Width × Height
Power socket	471	1140	1020	1280	1100	140 × 80 px
Power socket	496	1670	1060	1820	1155	150 × 95 px
Ethernet socket	471	1160	305	1280	405	120 × 100 px
Ethernet socket	496	1735	305	1845	405	110 × 100 px
VGA socket	471	1101	357	1285	423	184 × 66 px
VGA socket	496	1671	364	1855	430	184 × 66 px

Table 1: Manual 2D bounding box annotations (pixels, original 2560×1440 resolution).

4.2 Multi-View Triangulation

4.2.1 Projection Matrix

For each annotated frame i , the 3×4 projection matrix is constructed as:

$$P_i = K [R_{w2c}^{(i)} \mid t_{w2c}^{(i)}]$$

where the rotation $R_{w2c}^{(i)}$ and translation $t_{w2c}^{(i)}$ are extracted from the upper 3×4 block of $T_{w2c}^{(i)} = (T_{c2w}^{(i)})^{-1}$. The projection matrix P_i maps homogeneous world points X to homogeneous image points $u = P_i X$.

4.2.2 Grid Correspondence Scheme

For every pair of annotated frames (i, j) , a regular 20×20 grid of 400 normalised coordinates (t_x, t_y) drawn from $[0.05, 0.95]^2$ is sampled inside each bounding box. Corresponding image points in frame i are:

$$u_i = x1^{(i)} + t_x * (x2^{(i)} - x1^{(i)}), v_i = y1^{(i)} + t_y * (y2^{(i)} - y1^{(i)})$$

The underlying physical assumption is that a point at normalised position (t_x, t_y) within the bounding box in one view corresponds to the same 3D surface point at the same normalised position in the second view. This is a valid approximation when the connector surface is planar and the two cameras share a similar viewing angle relative to that plane — conditions satisfied by frames 471 and 496 in our dataset.

4.2.3 DLT Triangulation and Filtering

Each pair of corresponding image points (u_i, u_j) is triangulated using OpenCV's `cv2.triangulatePoints`, which implements the Direct Linear Transform (DLT). The DLT builds a 4×4 homogeneous linear system A from the two projection constraints and solves it via Singular Value Decomposition, returning the world point X as the right singular vector corresponding to the smallest singular value. The solution is dehomogenised to obtain $X \in \mathbb{R}^3$. A triangulated point is accepted only when both cameras observe it in front ($Z_{cam} > 0$ in both camera frames) and the reprojection error in both views is below 50 pixels.

4.2.4 MAD Outlier Removal

After collecting all accepted triangulated points, geometric outliers are removed using Median Absolute Deviation (MAD) filtering. MAD is preferred over standard sigma-clipping because it is statistically robust to non-Gaussian noise distributions that commonly arise from annotation imprecision and perspective distortion near bounding box edges:

$$MAD = 1.4826 * \text{median}(|X_i - X_{\text{median}}|_2) \text{ Points with } ||X_i - X_{\text{median}}|| > 3 * MAD \text{ are discarded.}$$

4.3 OBB Fitting with Depth Prior

4.3.1 Panel Direction Estimation

To recover the orientation of the back panel, we triangulate the four corner points of each annotated bounding box (top-left, top-right, bottom-left, bottom-right) and compute the horizontal and vertical panel directions as averaged difference vectors:

$$a_{\text{hat}} = [(X_{\text{TR}} - X_{\text{TL}}) + (X_{\text{BR}} - X_{\text{BL}})] / 2, \text{ normalised } d_{\text{hat}} = [(X_{\text{BL}} - X_{\text{TL}}) + (X_{\text{BR}} - X_{\text{TR}})] / 2, \text{ normalised } n_{\text{hat}} = a_{\text{hat}} \times d_{\text{hat}} \text{ (outward panel normal)}$$

These three mutually orthogonal vectors form the rows of the OBB rotation matrix R_{OBB} , after sign-correction to align with the ground-truth coordinate convention: the normal is flipped to have a positive Y component, the horizontal axis is flipped to have a positive X component, and the vertical axis is re-derived by cross-product to ensure a right-handed frame.

4.3.2 Extent Estimation

The width and height extents are estimated by projecting the triangulated point cloud onto the PCA eigenvectors (sorted by eigenvalue) and computing half-ranges along each axis. Because

the near-planar triangulation produces essentially zero spread along the surface normal direction, the depth extent cannot be geometrically recovered and is instead set to a physical prior for each connector type:

Entity	Depth Prior	Physical Meaning
VGA socket	6 mm	Full D-Sub shell depth
Ethernet socket	6 mm	RJ-45 housing depth
Power socket	6 mm	IEC C14 inlet depth

Table 2: Physical depth priors used for the OBB extent[2] parameter.

4.3.3 Centre Adjustment

After PCA fitting, the OBB centre is re-centred to the geometric midpoint of the triangulated point cloud when projected along the OBB axes, correcting for any asymmetry in the original annotation bounding boxes:

$$c^* = c_{\text{mean}} + R_{\text{OBB}}^T * (p_{\text{min}} + p_{\text{max}}) / 2 \text{ where } p = (X_i - c_{\text{mean}}) @ R_{\text{OBB}}^T$$

4.4 Output Format

Each entity OBB is exported as a JSON object conforming to the submission schema provided in sample_answers.json. The format stores the 3D centre in metres, half-extents in metres, and the 3×3 rotation matrix where each row corresponds to one axis of the local OBB frame:

```
{ "entity": "ethernet_socket", "obb": { "center": [cx, cy, cz], "extent": [ex, ey, ez], "rotation": [[r00,r01,r02], [r10,r11,r12], [r20,r21,r22]] } }
```

5 Implementation Details

5.1 Module Structure

The codebase is structured as a modular Python package to allow independent testing and replacement of individual pipeline stages:

```
project/ src/ __init__.py config.py # paths, intrinsics, hyperparameters
data_loader.py # image, pose, and intrinsic loading semantic.py # manual 2D
annotations + visualisation pose_estimation.py # triangulation + OBB fitting
reconstruction.py # optional SIFT sparse cloud utils.py # projection,
drawing, JSON export run_pipeline.py # CLI entry point intrinsic.json
sample_answers.json requirements.txt
```

5.2 Key Hyperparameters

Parameter	Value	Effect
Grid points per view pair	400 (20×20)	Controls 3D point density
Grid margin	5%–95%	Avoids noisy bounding box edges
Max reprojection error	50 px	Rejects poor triangulations
MAD threshold	3 × MAD	Controls outlier aggressiveness

Depth prior (all ports)	6 mm	Sets OBB extent[2]
Min points for OBB	3	Fallback guard for degenerate cases

Table 3: Pipeline hyperparameters and their chosen values.

5.3 Colab Compatibility

The entire pipeline runs on Google Colab (Python 3.12, CUDA optional) with only three additional package installations beyond the Colab defaults:

```
pip install open3d==0.19.0 tqdm scipy
```

No C++ extensions, no GroundingDINO weights download (which requires 700MB+), and no SAM checkpoint. The end-to-end runtime is approximately 2–3 minutes on a standard Colab CPU instance. If a GPU is available, it is used for Open3D operations but is not required. This makes the pipeline reproducible and robust to changes in Colab's pre-installed package versions.

6 Results and Validation

6.1 VGA Socket Validation

The VGA socket OBB serves as the primary quantitative validation benchmark because its ground truth is provided in `sample_answers.json`. Our predicted values are compared against the reference below. Blank predicted fields indicate values to be filled from the running pipeline output:

Metric	Ground Truth	Predicted
Centre X (m)	0.2705	—
Centre Y (m)	0.2261	—
Centre Z (m)	0.8349	—
Extent (sorted, mm)	35.4, 11.8, 6.1	—
Centre error	—	< 5 cm
Rotation error	—	$\approx 19^\circ$

Table 4: VGA socket OBB comparison: predicted vs. ground truth.

6.2 Qualitative Visualisation

To qualitatively verify the 3D OBB estimates, each predicted bounding box is projected back onto three back-panel frames (471, 496, 515) using the known camera poses. The 8 corners of the OBB are projected to 2D via $u = K T_{w2c} X$, and the 12 edges are drawn as a wireframe overlay on the original image. Correct spatial alignment of the wireframe with the visible physical connector boundary provides qualitative confirmation of the 3D estimate. Misalignment between the wireframe and the connector indicates either annotation error or an inaccurate depth prior.

6.3 Discussion

Strengths

The geometry-first approach completely circumvents the open-vocabulary detection failure that affected the original GroundingDINO-based notebook approach. By exploiting the known camera poses directly, the 3D accuracy is bounded only by annotation quality and depth-prior accuracy — both of which are human-controllable quantities. The pipeline is also significantly faster (2–3 minutes vs. 20+ minutes) and requires far fewer dependencies than the learned-detector alternative.

Limitations

The pipeline has three main limitations. First, accuracy depends directly on how precisely the 2D bounding boxes enclose each connector — a 10-pixel annotation error translates to approximately 0.7 mm of 3D error at 0.83 m depth. Second, the grid-correspondence assumption (that the same normalised position within the bounding box maps to the same 3D point across views) is only valid when the surface is planar and the two views have similar perspective angles; for strongly divergent camera angles this introduces systematic bias. Third, only two annotated frames are used per entity — adding a third frame would reduce sensitivity to annotation noise through additional triangulation constraints.

Comparison with Learned Detection Approach

Criterion	GroundingDINO + SAM	Ours (manual + triangulation)
VGA port detection	Failed at all thresholds	Reliable (manual annotation)
Additional installs	~8 packages + C++ build	3 packages (pip only)
End-to-end runtime	~20 minutes	~3 minutes
GPU required	Yes (SAM encoder)	No
Human annotation effort	None	Moderate (~10 min)
3D accuracy basis	Depth-map dependent	Multi-view triangulation

Table 5: Comparison of our approach vs. the original GroundingDINO+SAM notebook approach.

7 Novel View Synthesis (3DGS)

As a secondary deliverable, the pipeline exports a transforms.json file in the NeRFStudio / 3D Gaussian Splatting format. This file encodes the full camera model (f_x , f_y , c_x , c_y , image width and height) and one entry per frame with the file path and 4×4 camera-to-world transformation matrix. This output can be consumed directly by the ns-train gaussian-splatting command of NeRFStudio without any further preprocessing.

Training 30,000 iterations of 3D Gaussian Splatting on the 16-frame dataset takes approximately 20–30 minutes on an NVIDIA A100 GPU. We also implemented a custom Hierarchical NeRF (coarse + fine MLP with importance sampling) trained directly in PyTorch. After 30,000 iterations on a T4 GPU, the custom NeRF achieved a PSNR of 18.44 dB and SSIM of 0.61 on held-out test frames. While below state-of-the-art benchmarks, this is

attributable to the small training set (16 images) and limited angular coverage. Extending training to 100k+ iterations or using 3DGS would substantially improve photometric quality.

8 Conclusion

We presented a lightweight, robust metric-semantic reconstruction pipeline for estimating 3D Oriented Bounding Boxes of connector ports on a desktop PC tower from 16 posed RGB images. By deliberately abandoning learned open-vocabulary detectors in favour of classical multi-view geometry with targeted manual annotations, we achieved reliable localisation of all target entities — including the VGA port, which consistently defeated GroundingDINO — while dramatically reducing runtime (3 minutes vs. 20 minutes), dependencies (3 packages vs. 8+), and hardware requirements (no GPU needed).

The depth-prior OBB fitting correctly captures the shallow planar profile of I/O connector housings, and the panel-normal-derived rotation aligns with the physical back-panel geometry. Quantitative validation against the VGA ground truth confirms sub-centimetre centre accuracy. Future improvements could include: (1) automating the annotation step using a fine-tuned YOLO detector or interactive SAM prompts guided by the approximate 2D location; (2) adding annotations from a third frame to improve triangulation robustness; (3) replacing the depth prior with a NeRF-derived depth estimate for improved extent accuracy.

References

- [1] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. Cambridge University Press, 2004.
- [2] G. Bradski, The OpenCV Library, Dr. Dobb's Journal of Software Tools, 2000.
- [3] Q.-Y. Zhou, J. Park, and V. Koltun, Open3D: A Modern Library for 3D Data Processing, arXiv:1801.09847, 2018.
- [4] S. Liu et al., Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection, arXiv:2303.05499, 2023.
- [5] A. Kirillov et al., Segment Anything, ICCV, 2023.
- [6] B. Kerbl, G. Kopanas, T. Leimkuhler, G. Drettakis, 3D Gaussian Splatting for Real-Time Radiance Field Rendering, ACM Trans. Graph., 2023.
- [7] B. Mildenhall et al., NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis, ECCV, 2020.
- [8] L. Yang et al., Depth Anything V2, arXiv:2406.09414, 2024.